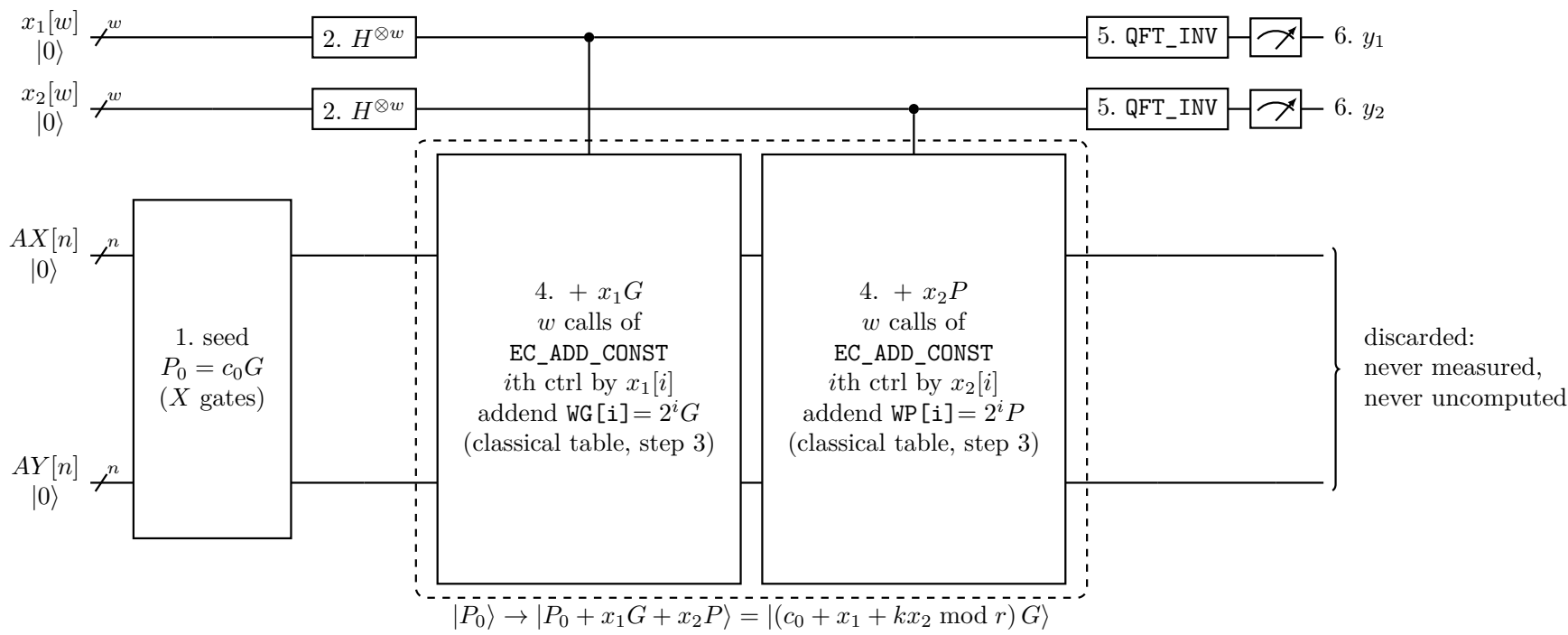
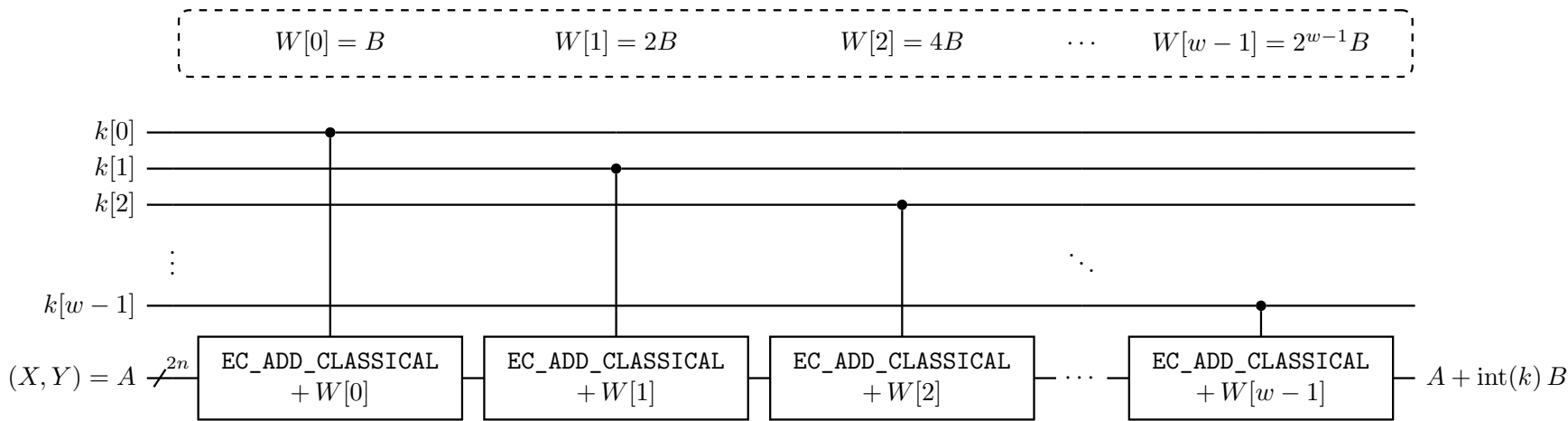


The whole algorithm, and how the layers connect



PRECOMPUTE_POWER: the doubling chain $W[i] = 2^iB$ is built on the host at circuit-build time — $w - 1$ classical inversions, 0 qubits, **0 gates in the circuit**



Reversibility is the cost driver

Roughly half the work at every layer is undoing, and four instances follow: modular addition spends three of its five adders returning two flags to $|0\rangle$; the inversion's $2n$ -bit branch history clears only by running the inversion backwards, so the caller pays twice; point addition's four inversion calls contribute no bit to the output; and the AND gadget uncomputes for zero T, which is what makes the other three affordable.

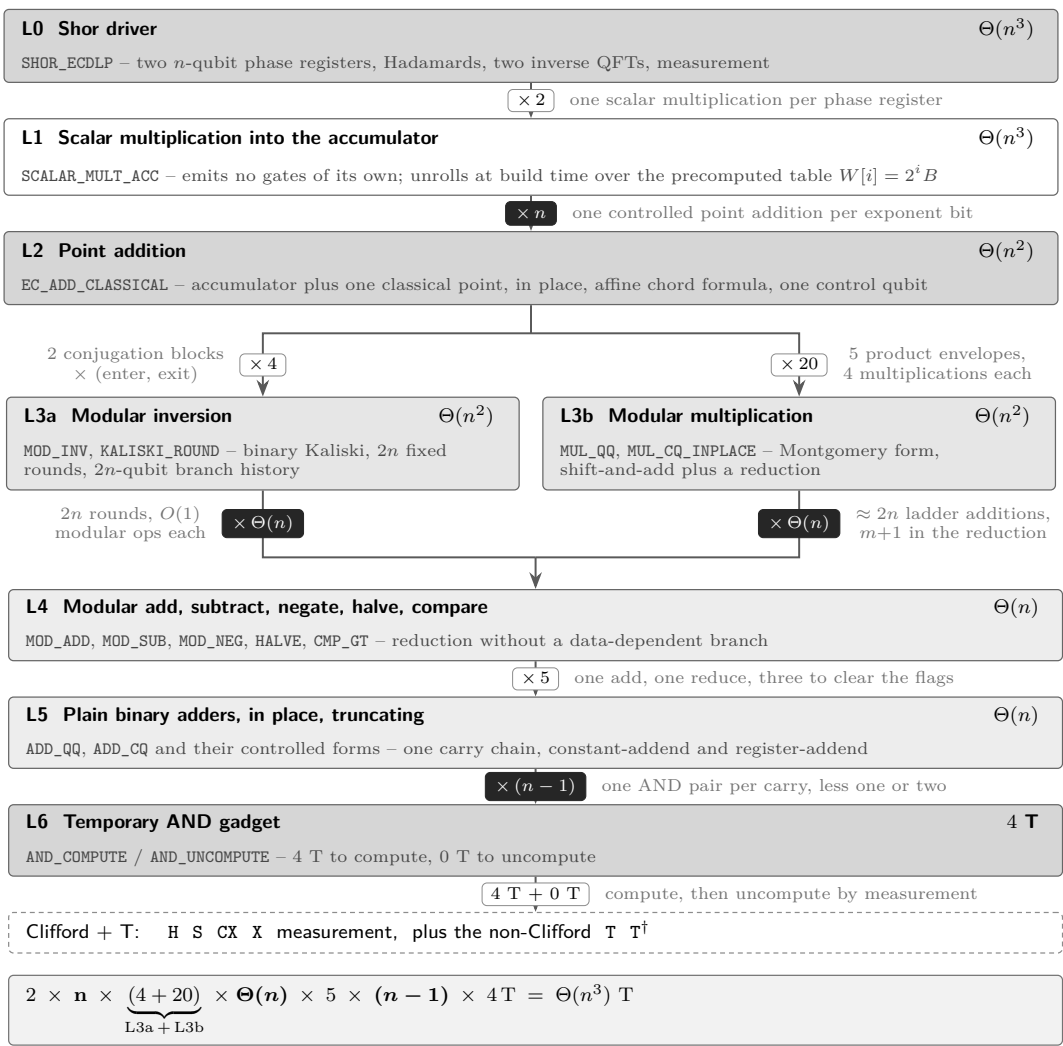
The call chain (the stack, upper right)

caller	callee	calls
driver	scalar multiplication	2
scalar multiplication	point addition	n
point addition	modular inversion / multiplication	4 / 20
inversion	modular add, subtract	$4n$
multiplication / modular add	plain adder	$\approx 3n / 5$
plain adder	AND gadget	$n - 1$

Multiply along any one path: $\Theta(n^3)$ T gates, about 5.8×10^{10} at $n = 256$. Scalar multiplication emits no gates of its own -- it is a build-time unroll over the table (the middle left diagram).

Interface. Every boundary passes little-endian registers holding plain residues in $[0, p)$ -- no Montgomery factor, no infinity flag -- with all ancillas at $|0\rangle$ on entry and exit.

Qubits. $4n$ persistent: two exponent registers plus two accumulator coordinates. One addition is live at a time, adding a slope register, a $2n$ -bit history, one product scratch and the inversion's own $3n$. Plan $10n$ to $11n$; at toy sizes the additive constants dominate.



The problem

$E : y^2 = x^3 + ax + b$ over $\mathbb{F}_p$, $n = \text{bitlen}(p)$. The base point G has prime order r ; the public point is $P = kG$. Recover k .

Phase estimation in two dimensions (upper left diagram)

The map $f(x_1, x_2) = x_1G + x_2P$ hides the subgroup generated by $(-k, 1)$, so Fourier sampling it yields k . Hadamard two exponent registers of width w . Seed the accumulator AX, AY with a classical $P_0 = c_0G$, c_0 random in $[1, r - 1]$: $(0, 0)$ is an off-curve point, not the identity, and small c_0 makes early additions exceptional on half the branches. Then $2w$ controlled point additions leave the accumulator holding $(c_0 + x_1 + kx_2 \bmod r)G$; inverse-QFT and measure both exponent registers. Never uncompute the accumulator: the key lives in its *correlation* with the exponents, so reversing would restore $|P_0\rangle$ on every branch and return $(0, 0)$ with certainty. Discarding halves the circuit.

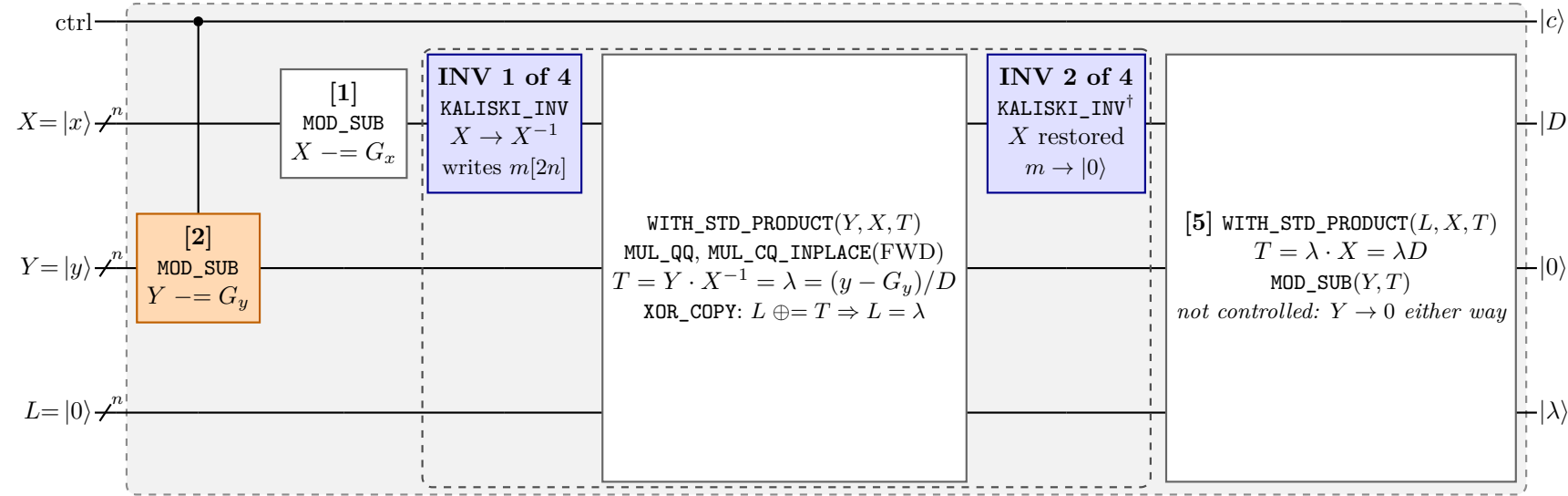
Key recovery. The QFT resolves only phases y/N , $N = 2^w$, while the physical phases are u/r and $r \nmid N$ -- so rescale: $u_i = \text{round}(y_i r / N) \bmod r$, exact whenever $N \geq r$. Reject $u_1 = 0$ (probability $1/r$, no information); otherwise $k = u_2 u_1^{-1} \bmod r$, checked against $kG = P$. Success is 0.7 to 0.8 per shot, so ten shots suffice. Size $w = \lceil \log_2 r \rceil + 2$ to 4 by r , not p ; that is n on a good curve, assumed below.

Two structural decisions

The base point is classical. Since $x_1G = \sum_i (x_1)_i 2^i G$ and the $2^i G$ are independent of x_1 , the host builds both doubling tables before a gate is emitted: no point is ever doubled on the quantum computer and no addition has two quantum summands. The price: only the generic chord formula is implemented, so a run fails if an addend shares an x -coordinate with the accumulator -- $O(n/r)$ with a random seed, dominant at toy sizes.

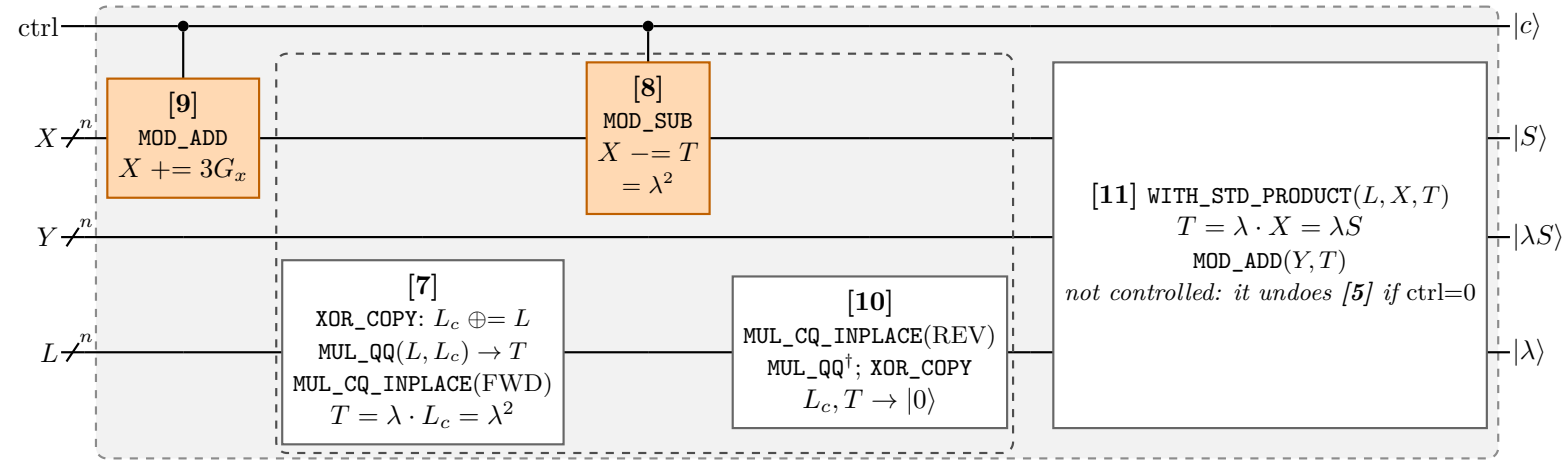
Addition is in place. Each addition's output feeds the next. Out of place costs either $O(n^2)$ qubits or double the gates to uncompute inputs; in place pins the accumulator at $2n$.

Point addition: the centrepiece



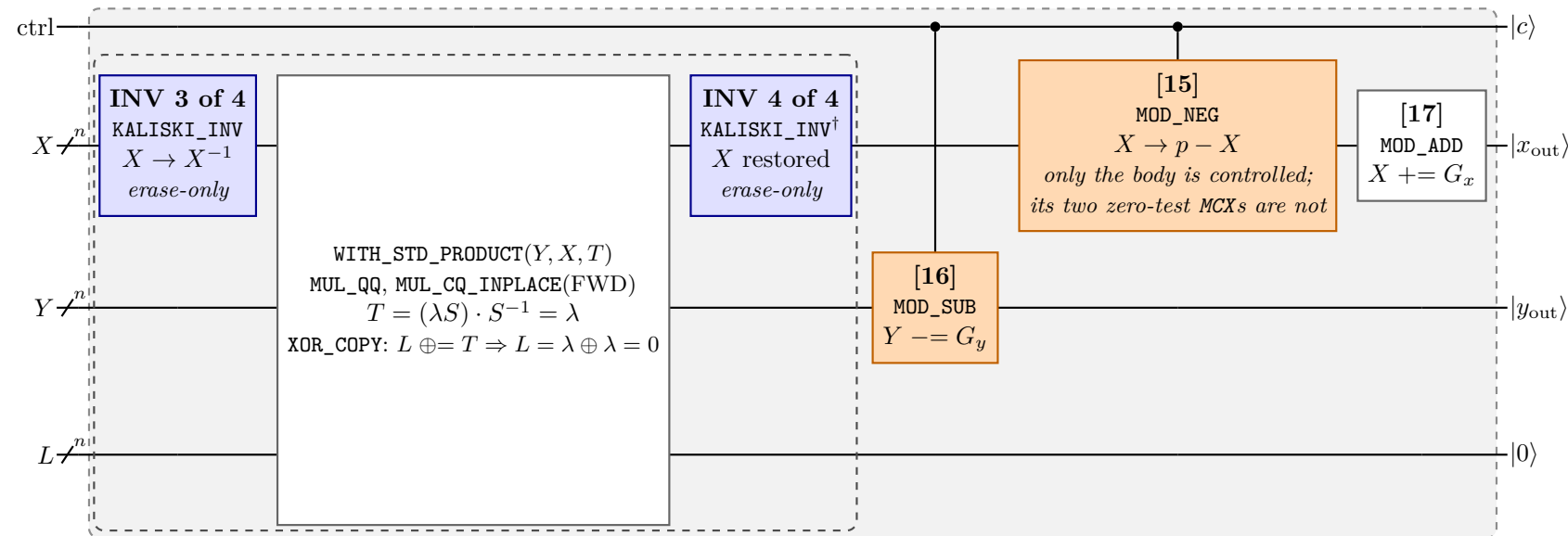
movement 1 — shift by the classical constants, build the slope, drive Y to zero; $D := (x - G_x) \bmod p$
the dashed bracket is one **LAMBDA_XOR** call, steps [3,4,6]: a **KALISKI_INV** conjugation round its body

orange carries the control qubit — five blocks in the whole routine blue **KALISKI_INV** — four calls, two of them only to erase L



movement 2 — move X to $S := (G_x - x_{\text{out}}) \bmod p$ and rebuild Y

the dashed bracket is the squaring envelope, steps [7], [8], [10]: it opens and closes unconditionally; only [8] is controlled



movement 3 — erase the slope, then finish both coordinates; the dashed bracket is **LAMBDA_XOR** again, steps [12,13,14], re-run on the output point

INV 3 and **INV 4** do no arithmetic on the point at all: they exist only to return L to $|0\rangle$, at close to half the cost of the routine

Interface

In: AX, AY , n qubits each, holding an affine accumulator $A = (x, y)$; one control qubit; and a classical addend $C = (C_x, C_y)$ compiled in. **Out:** AX, AY hold $A + C$ if the control is 1, unchanged bit-for-bit if 0; control untouched; every ancilla back to $|0\rangle$ — no output register, no garbage.

Preconditions, on every branch: $x \neq C_x$, and $\lambda^2 - x - C_x \neq C_x$. The value p itself is illegal in either register. **Above:** the scalar-multiplication loop calls this once per exponent bit — $C = 2^i G$ at the first call site, $2^i P$ at the second, $2n$ calls per run. **Below, per call:** 4 inversions, 10 quantum-quantum multiplications, 10 in-place constant multiplications, 8 modular add/subtracts, 1 modular negation, 4 CNOT copies. Peak private width $4n$.

The identity that makes it in place

$\lambda = (y - C_y)/(x - C_x)$, $x_{\text{out}} = \lambda^2 - x - C_x$, $y_{\text{out}} = \lambda(x - x_{\text{out}}) - y$. The last form cannot run in place: it wants the *input* x , gone by the time y_{out} is written. But $\lambda(x - C_x) = y - C_y$ gives $\lambda x - y = \lambda C_x - C_y$, so

$$y_{\text{out}} = (\lambda x - y) - \lambda x_{\text{out}} = \lambda(C_x - x_{\text{out}}) - C_y.$$

Same number, different circuit: only classical constants and x_{out} , which AX already holds.

The three movements (left to right on the diagram)

Write $D = x - C_x$ and $S = C_x - x_{\text{out}} = D + 3C_x - \lambda^2$.

One -- build the slope, drive AY to zero. Subtract C_y from AY and C_x from AX ; an inversion conjugation round a quantum-quantum product XORs $AY \cdot AX^{-1} = \lambda$ into an empty L ; then $AY -= \lambda AX$. Since $\lambda D = y - C_y$ identically, AY hits *exactly* zero — nothing ever holds y_{out} . State $(D, 0, \lambda)$.

Two -- move AX , rebuild AY . $AX += 3C_x$, $AX -= \lambda^2$, leaving S ; then $AY += \lambda AX$. State $(S, \lambda S, \lambda)$.

Three -- erase the slope. Re-run the movement-one slope circuit on the *output* point: $AX = S$ and $AY = \lambda S$, so the recomputed quotient is $\lambda S/S = \lambda$ and the XOR returns L to $|0\rangle$ — the XOR is bitwise and both operands are the same integer in $[0, p)$. Then $AY -= C_y$, negate AX , add C_x .

The erasure is not forced by algebra: it works because the routine made AX the very factor multiplying λ inside AY — what the rewritten y_{out} bought. The price is the second inversion pair, near half the routine: four inversions, not two, each dragging a $2n$ -bit history.

Squaring needs a second copy: XOR L into L_c , multiply, XOR back. The multiplier declares two *distinct* operand registers, so an aliased call risks a silently wrong product rather than an error; a dedicated squarer removes L_c .

The control optimisation

Five $\Theta(n)$ blocks carry the control: both C_y shifts of AY , $AX += 3C_x$, $AX -= \lambda^2$, and the negation of AX — only its body; the two zero-test MCXs stay uncontrolled, safe since a skipped body leaves AX unchanged and the second MCX still clears the flag. Everything $\Theta(n^2)$ is emitted bare.

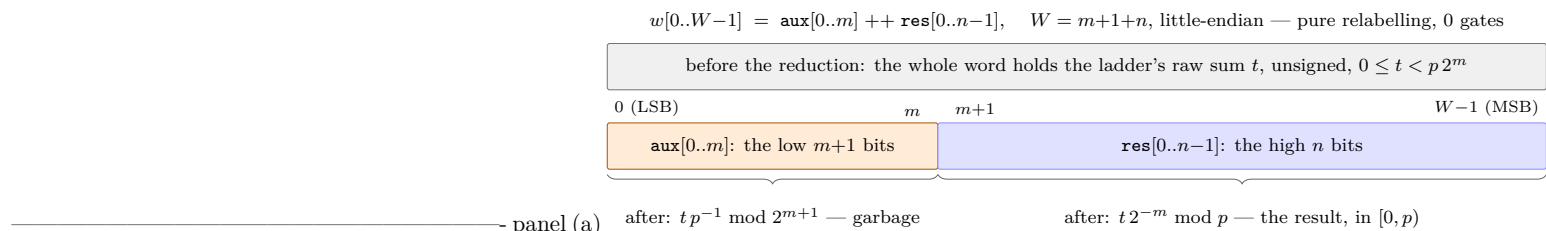
With the control off, the two uncontrolled constant shifts of AX bracket the routine and cancel, every other net update to AX being controlled. The two $AY \mp L \cdot AX$ steps then see the *same* L and the *same* AX , so they compose to the identity — the crux, turning two would-be controlled multiplications free. The second slope block recomputes whatever the first stored, so L clears either way. Wrapping the whole call in a control is correct but touches $\Theta(n^2)$ gates and roughly doubles the T-count.

Exceptional cases

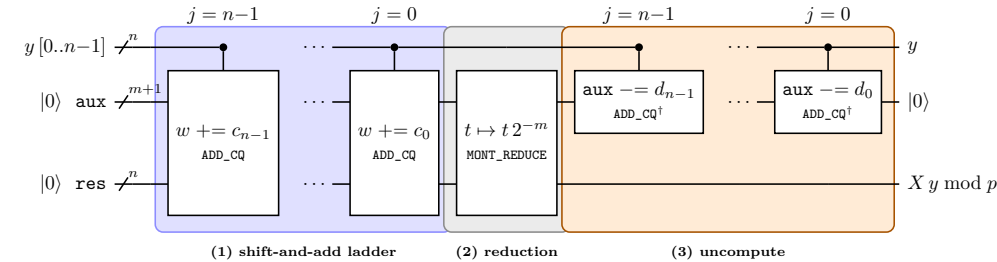
Both divide by zero inside an inversion, which maps $0 \mapsto 0$ rather than faulting: the circuit completes and lies. (a) $x = C_x$, i.e. $A = \pm C$: coordinates garbage, L freed non-zero and entangled. (b) $x_{\text{out}} = C_x$, i.e. $A = -2C$: coordinates *correct*, only L leaks — invisible to any value-level test.

The bad accumulator values are $\pm C$ and $-2C$: three points out of $\approx p$, so about $3 \cdot 2^{-n}$ per addition and $6n \cdot 2^{-n}$ per run, near 2^{-245} at $n = 256$. On toy curves they dominate: over $\mathbb{F}_7$ three of nine affine points are exceptional, and a six-addition run hits one with probability 0.91. Test on hand-chosen chains excluding $\pm C$ and $-2C$, and assert on the ancillas, not only the coordinates.

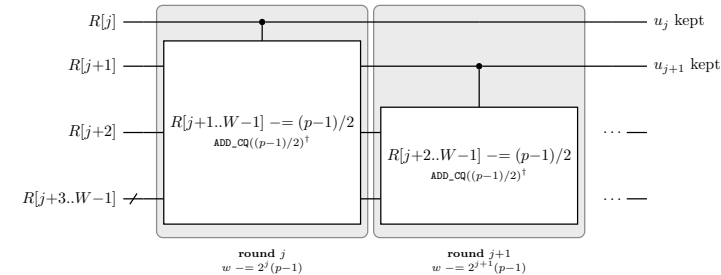
Montgomery multiplication and modular addition



(a) the work register. The order is forced: the reduction consumes quotient digits from the bottom of the word and leaves its garbage there, and deposits the reduced value at the top.



(b) $\text{MUL_CQ}(X, y, p, m, \text{res})$. (1) for $j = n-1..0$: with $\text{ctrl}(y[j])$: $\text{ADD_CQ}(c_j, w[0..W-1])$ — n plain adders at full width, and nothing inside the loop is reduced. (2) $\text{MONT_REDUCE}(w[0..W-1], p, m)$. (3) for $j = n-1..0$: with $\text{ctrl}(y[j])$: with inv: $\text{ADD_CQ}(d_j, \text{aux}[0..m])$ — the same ladder, truncated to $m+1$ bits. Compile-time constants: $c_j = (X 2^m 2^j) \bmod p$, pre-reduced, and $d_j = (c_j p^{-1}) \bmod 2^{m+1}$. Pre-reduction bounds $t \leq n(p-1)$, so $m = \min\{m : n(p-1) < p 2^m\} \leq \lceil \log_2 n \rceil$: fixed by the *count* of partial products, never by the size of X .



(c) inside MONT_REDUCE : two consecutive estimation rounds. The control is the wire itself, so the test is free, and the subtracted multiple of $p-1$ is even, so the quotient digit u_j survives where the control read it. The target window climbs one wire per round — that slide *is* the division by two. After all m rounds: correction, n swaps, fold.

Montgomery multiplication (the tall left-hand diagram)

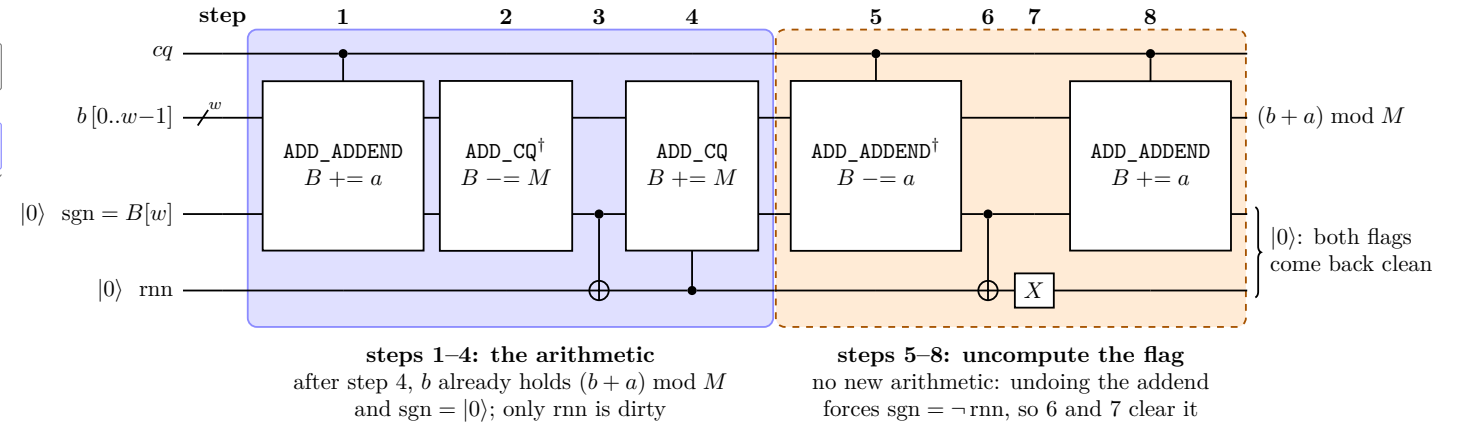
Why not schoolbook. Reducing after each partial product spends a controlled modular addition per operand bit -- five ripples and two flags, some $52n^2$ T for the product.

The idea. Ordinary reduction asks *is this at least p?* -- an answer buried in the carry chain, unobtainable without a ripple. Montgomery asks *is this odd?* -- an answer sitting on wire 0, free to read and a legal control. Because p is odd the right multiple of p always clears that bit, and that multiple is of the *even* $p-1$, so the digit survives where the control read it, unremembered. The halving is a slice, not a gate: the next addition targets one wire higher. The accumulator may exceed p throughout, and reduction is paid ONCE, in $m+1$ additions.

Representation contract. A register carries a compile-time shift μ , emitting no gates: raw bits = value $\cdot 2^\mu \bmod p$. Shifts add under multiplication and each reduction subtracts its own m , so $\mu_{\text{out}} = \mu_x + \mu_y - m$, cancelled by the caller's pre-encoding by 2^m . Unequal shifts cannot be added: make it a compile-time error, the only repair being bitwise decomposition at $5n$ ripples in place of five.

The three stages. (1) Plain, non-modular controlled adders into a work word, aux in the low $m+1$ wires and the result in the high n -- the order the reduction forces, eating digits from the bottom up. (2) m controlled additions of $(p-1)/2$ into a window climbing one wire per round, a correction under the sign, n swaps, one CNOT. (3) That CNOT makes the low-order garbage exactly $t p^{-1} \bmod 2^{m+1}$, *linear* in the raw sum t , so aux clears by re-running the ladder truncated to $m+1$ bits instead of inverting the reduction. Unfolded it would be the nonlinear quotient-digit sequence, and aux dirty forever.

Interface. Five standard-form windows per point addition emit ten quantum-by-quantum products (each forward and inverted) and ten in-place constant multiplications; the four inversions call the constant form twice each, charged to them. Downward, $2n+2m+1$ adder calls for a quantum product, $2n+m+1$ for a constant one. A quantum product forces $m = \min\{m : (p-1)^2 < p 2^m\}$, essentially n : some $20n^2$ T. A constant one pre-reduces its addends, so $m \leq \lceil \log_2 n \rceil$ and about $4n^2$ T. The reduction and both conversion constants must use that same m .



Modular addition (the upper right diagram)

Interface. Point addition calls it eight times and its negation once; each of the inversion's $2n$ rounds calls it twice. Downward, five plain truncating adders. Across the boundary: a w -qubit little-endian target holding a reduced residue ($w = n$ at $M = p$, $w = n+1$ at $M = 2p$), an equally reduced constant or register addend, one optional control.

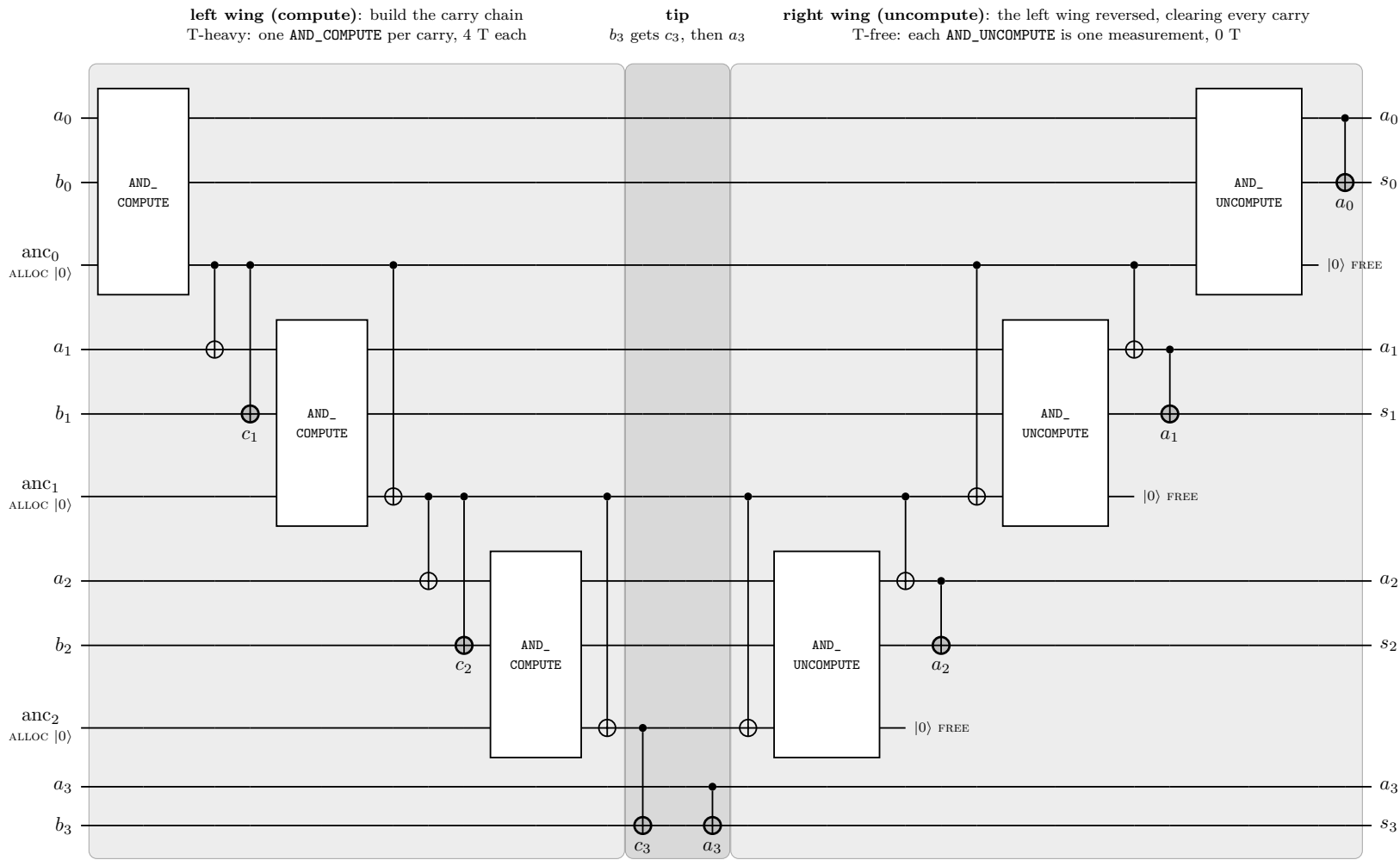
The difficulty. The sum is one adder; reduction is the rest of the circuit. Whether to subtract M is one bit, and a reversible circuit cannot produce a bit and drop it: left anywhere it records the operands, and phase estimation interferes only because no such record survives. Nor is it readable off the result -- both branches land in $[0, M)$.

The schedule. A sign qubit above the target makes a $(w+1)$ -qubit two's-complement word; every adder acts on all of it and truncates mod 2^{w+1} , so modularity lives in the schedule, not the adder. Add a , subtract M : the word holds $a+b-M \in [-M, M-2]$, in range with no double wrap, so $\text{sgn} = 1$ iff no reduction is needed. A CNOT latches it into rnn ; re-adding M under rnn finishes the arithmetic and clears sgn . Subtracting a back leaves b if no reduction fired, $b-M$ if one did: $\text{sgn} = \neg \text{rnn}$ *unconditionally*, the branches having swapped flags. A second CNOT therefore sets rnn to 1 either way, an X clears it, a fifth adder restores the answer. **Silent failure.** An unreduced target collapses the range argument: rnn is freed at $|1\rangle$ and a dirty qubit re-enters the allocator. At $b = M$ the answer is right for every a while the flag is always dirty -- the value test passes, the circuit fails elsewhere. Negation hits this at input 0; hence its is-zero guard.

The neighbours. Subtraction is these eight steps daggered. Negation is X on every qubit, a bare adder of $M+1$, then one modular addition.

Shift and compare. Doubling and halving are cyclic SWAP ladders: zero T, exact when the wrapping bit is $|0\rangle$. Comparison keeps the borrow of a discarded subtraction; budget it twice for the conjugation sandwich. About $20n$ T per modular addition.

Adders, the temporary AND, and the cost roll-up



The ripple adder (the V-shaped diagram at left)

Carries obey $c_{i+1} = ((a_i \oplus c_i) \wedge (b_i \oplus c_i)) \oplus c_i$ -- majority rewritten so the sole nonlinearity is one AND of values already on the wires. Hence the V: a wing rippling carries up at 4 T per bit, a tip, a wing walking back down clearing them at zero T.

The structural point. The uncompute wing must see the control values the compute wing saw, so the CNOT stripping c_i off the ancilla runs *before* it and the restore of **a**[i] *after*. Deliberately unmirrored is **CX(anc[i-1] -> b[i])**: not scratch but the sum's carry contribution, so the CNOTs that survive are exactly the sum bits.

Constant addend. The **a** register and every gate touching it vanish. The AND wants $a_i \oplus c_i$, which is the previous carry ancilla read active-high or active-low by a compile-time bit -- no gate computes it. Saves n qubits, an AND pair, $3n$ CNOT.

Controlled forms. Never wrap the block. With a register addend the wings are exact inverses on **a** and **anc** and cancel when the control is off, so only the unmirrored writes take a control -- each an **AND_COMPUTE**; **CX**; **AND_UNCOMPUTE** sandwich at 4 T on one reused scratch qubit. The constant-addend form controls the seed and the polarity flips; zero then propagates down the carry chain, leaving the sum CNOTs as no-ops.

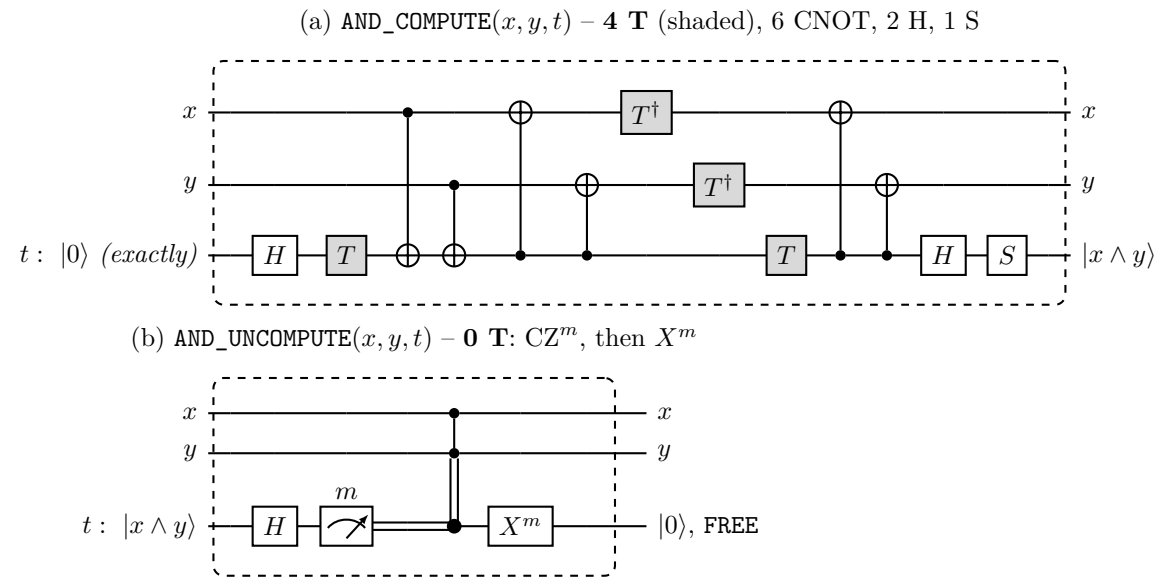
$n \geq 2$	QQ	QQ ctrl	CQ	CQ ctrl
T	$4(n-1)$	$8n$	$4(n-2)$	$4(n-2+A_0)$
ancilla	$n-1$	n	$n-1$	$n-1$

The Fourier alternative shows zero T only because its arbitrary-angle rotations still need synthesising at about $3 \log_2(1/\epsilon)$ T each: 10^7 T at $n = 256$ against 10^3 here. Frameworks default to it; pass the ripple adder explicitly.

routine	T per call	multiplicity
AND pair	4	per carry bit; 84% of all T
Montgomery multiply	$20n^2$ QQ, $4n^2$ CQ	$\approx 3n$ adders
inversion	$230n^2 + 364n + 2$	$34n$ adders
point addition	1.13×10^8 at $n = 256$	4 inversions, 20 multiplies
full run	$\Theta(n^3)$: 5.8×10^{10} at $n = 256$	$2n$ point additions

That is 2.5×10^{11} total gates over $10n$ to $11n$ logical qubits, with the QFT rotations unsynthesised: an order of magnitude, not a target.

Build bottom-up. Most routines here permute basis states: simulate the gate list classically over all (a,b) to seven bits and both control values, with **FREE** an assertion, not a deallocation -- which proves ancilla cleanliness in the same sweep. Basis states miss phase, so also run both AND controls, and a point addition's control, in $|+\rangle$: a dropped CZ shows only as a sign on the $|11\rangle$ branch.



The interface at the bottom of the stack

Who calls the adder. Five calls per modular add/subtract, about $3n$ per Montgomery multiplication, 17 per Kaliski round -- $34n$ per inversion -- and 4 inversions plus 20 multiplications per point addition, $2n$ point additions per run. **The adder calls only the AND gadget:** $n-1$ pairs plain, $2n$ controlled.

What crosses. Two little-endian n -qubit registers of plain integers, non-aliased, $n \geq 2$, no modulus. $b \leftarrow (b+a) \bmod 2^n$ in place, truncating, a preserved. A caller needing the carry out widens b by a qubit; the modular wrapper always does. Ancillas are internal, returned $|0\rangle$.

The temporary AND

A Toffoli costs 7 T, but in an adder it only writes a carry into a fresh ancilla cleared a few gates later: the target is always exactly $|0\rangle$ and the write always undone. Demanding correctness on that subspace alone is cheaper.

Compute, thirteen gates -- 4 T, 6 CNOT, 2 H, 1 S, no ancilla:

H(t); T(t); CX(x->t); CX(y->t); CX(t->x); CX(t->y); TDG(x); TDG(y); T(t); CX(t->x); CX(t->y); H(t); S(t)

The four middle CNOTs are two pairs of *opposite* direction; reverse one and you get a different, still unitary gate. With $k = x + 2y + 4t$ as the basis label it equals $\text{CCX} \cdot \text{diag}(1, 1, 1, 1, i, -i, -i, -i)$, diagonal first, so the entries are indexed by the *input*: on $t = 0$ inputs the amplitude is exactly +1, and all the cheating sits on the excluded $t = 1$ columns. A dirty target leaves an (x,y) -dependent phase invisible to value tests.

Uncompute is H(t); m := MEASURE(t); if m == 1: { CZ(x,y); X(t) } -- zero T. The pre-H state is $\sum \alpha_{xy} |x,y\rangle |x \wedge y\rangle$; the H makes the last factor $(|0\rangle + (-1)^{x \wedge y} |1\rangle) / \sqrt{2}$, so each outcome has probability exactly 1/2 whatever the α , the ancilla disentangles either way, and the residue is exactly $\text{CZ}(x,y)^m$: Clifford, self-inverse, undone with certainty. Not repeat-until-success -- no failure branch, two successes differing by a known Clifford, and a compiler may absorb the correction into the Pauli frame.

Valid only as a matched pair on one triple, with x and y touched only diagonally between them. Compute alone is not a CCX; uncompute alone is not a unitary.